SQL PRACTICE HANDOUT

WHERE • GROUP BY • HAVING • ORDER BY • SQL Server

Learning goal

Practice filtering rows, grouping data, filtering grouped results, sorting output, and combining all four clauses. Try the questions before checking the answer key.

1. Create the Employee table

```
CREATE TABLE Employee (
    EmployeeID INT,
    EmployeeName VARCHAR(50),
    Department VARCHAR(30),
    Salary INT,
    City VARCHAR(30),
    Age INT
);
```

2. Insert sample data

```
INSERT INTO Employee
(EmployeeID, EmployeeName, Department, Salary, City, Age)
VALUES
(1, 'Arun', 'IT', 50000, 'Chennai', 25),
(2, 'Priya', 'HR', 40000, 'Bangalore', 38),
(3, 'Karthik', 'IT', 60000, 'Chennai', 38),
(4, 'Divya', 'Finance', 45000, 'Hyderabad', 27),
(5, 'Rahul', 'HR', 35000, 'Chennai', 24),
(6, 'Sneha', 'IT', 70000, 'Bangalore', 29),
(7, 'Vijay', 'Finance', 55000, 'Chennai', 35),
(8, 'Meena', 'HR', 48000, 'Hyderabad', 31),
(9, 'Ajay', 'IT', 40000, 'Hyderabad', 32),
(10, 'Kavya', 'Finance', 60000, 'Bangalore', 28),
(11, 'Suresh', 'IT', 55000, 'Chennai', 41),
(12, 'Anitha', 'HR', 52000, 'Bangalore', 26),
(13, 'Manoj', 'Finance', 38000, 'Chennai', 30),
(14, 'Lakshmi', 'IT', 65000, 'Hyderabad', 34),
(15, 'Deepa', 'HR', 42000, 'Chennai', 29);
```

3. Quick reference

Clause	Purpose	Example
WHERE	Filters individual rows before grouping.	WHERE Salary > 50000
GROUP BY	Groups rows that share values.	GROUP BY Department
HAVING	Filters groups after aggregation.	HAVING COUNT(*) > 3
ORDER BY	Sorts the result.	ORDER BY Salary DESC

Remember: WHERE filters rows; HAVING filters groups. Use WHERE before GROUP BY and HAVING after GROUP BY.

4. Practice questions

Questions 1–6: WHERE and ORDER BY

1. Display employees whose salary is greater than 50,000. [WHERE]

2. Display employees who work in the IT department. [WHERE]

3. Display employees who live in Chennai and earn more than 40,000. [WHERE]

4. Display employees whose age is between 25 and 35, inclusive. [WHERE]

5. Display all employees ordered by salary from highest to lowest. [ORDER BY]

6. Display all employees ordered by city alphabetically and then salary from highest to lowest. [ORDER BY]

7. Find the total salary paid by each department. [GROUP BY]

8. Find the number of employees in each city. [GROUP BY]

9. Find the average salary of employees in each department. [GROUP BY]

10. Display departments that have more than 3 employees. [HAVING]

11. Display cities where the total salary is greater than 150,000. [HAVING]

12. Display departments whose average salary is greater than 50,000. [HAVING]

13. Find the total salary of IT employees and show results from highest total salary to lowest. [WHERE + GROUP BY + ORDER BY]

14. Find the number of employees in each department whose individual salary is at least 45,000. [WHERE + GROUP BY]

15. Display departments having at least 3 employees, sorted by average salary from highest to lowest. [GROUP BY + HAVING + ORDER BY]

16. Find the total salary for each city, considering only employees aged 30 or above. Show the highest total first. [WHERE + GROUP BY + ORDER BY]

17. Display cities having more than 2 employees earning over 40,000. [WHERE + GROUP BY + HAVING]

18. Find average salary by department for employees aged above 25. Show only departments with average salary above 45,000, sorted by average salary descending. [All four clauses]

5. Answer key

SQL Server answers for all 18 practice questions

Question 1

```
SELECT * FROM Employee WHERE Salary > 50000;
```

Question 2

```
SELECT * FROM Employee WHERE Department = 'IT';
```

Question 3

```
SELECT * FROM Employee WHERE City = 'Chennai' AND Salary > 40000;
```

Question 4

```
SELECT * FROM Employee WHERE Age BETWEEN 25 AND 35;
```

Question 5

```
SELECT * FROM Employee ORDER BY Salary DESC;
```

Question 6

```
SELECT * FROM Employee ORDER BY City ASC, Salary DESC;
```

Question 7

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Department;
```

Question 8

```
SELECT City, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY City;
```

Question 9

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department;
```

Question 10

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY Department
HAVING COUNT(*) > 3;
```

Question 11

```
SELECT City, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY City
HAVING SUM(Salary) > 150000;
```

Question 12

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

Question 13

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employee
WHERE Department = 'IT'
GROUP BY Department
ORDER BY TotalSalary DESC;
```

Question 14

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employee
WHERE Salary >= 45000
GROUP BY Department;
```

Question 15

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY Department
HAVING COUNT(*) >= 3
ORDER BY AverageSalary DESC;
```

Question 16

```
SELECT City, SUM(Salary) AS TotalSalary
FROM Employee
WHERE Age >= 30
GROUP BY City
ORDER BY TotalSalary DESC;
```

Question 17

```
SELECT City, COUNT(*) AS EmployeeCount
FROM Employee
WHERE Salary > 40000
GROUP BY City
HAVING COUNT(*) > 2;
```

Question 18

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
WHERE Age > 25
GROUP BY Department
HAVING AVG(Salary) > 45000
ORDER BY AverageSalary DESC;
```

6. SQL logical order of execution

Order	Clause	What it does
1	FROM	Identifies source table(s).
2	ON / JOIN	Matches and combines rows from tables; part of the FROM operation.
3	WHERE	Filters individual rows.
4	GROUP BY	Creates groups.
5	HAVING	Filters groups.
6	SELECT	Chooses output columns and calculates expressions.
7	DISTINCT	Removes duplicate result rows.
8	ORDER BY	Sorts the result.
9	TOP	Limits rows returned in SQL Server; shown here as a simplified teaching sequence.

Example combining the clauses

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employee
WHERE Salary > 30000
GROUP BY Department
HAVING AVG(Salary) > 45000
ORDER BY AverageSalary DESC;
```

Processing idea: read the table → filter rows → group by department → filter groups → choose output → sort results.

SELECT aliases

In SQL Server, WHERE is processed before SELECT, so you generally cannot use a SELECT alias in WHERE. ORDER BY can usually use the alias because it is processed later.

```
-- Generally invalid in SQL Server
SELECT Salary * 12 AS AnnualSalary
FROM Employee
WHERE AnnualSalary > 600000;

-- Correct
SELECT Salary * 12 AS AnnualSalary
FROM Employee
WHERE Salary * 12 > 600000;

-- Alias works in ORDER BY
SELECT Salary * 12 AS AnnualSalary
FROM Employee
ORDER BY AnnualSalary DESC;
```

Memory tip

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

Note: The logical order is a teaching model. SQL Server's optimizer may choose a different physical execution plan.